A Survey on Robustness Distributions and their use for Neural Network Verification

Andrew Spiro Ivar van der Spoel Sander Honig

Seminar Trustworthy Artificial Intelligence

1 Introduction

This survey paper reviews a tool for the evaluation of adversarial robustness in neural networks (NNs): Robustness Distributions (RDs). RDs provide insights into the adversarial robustness of NNs which are not apparent from robustness metrics like robust accuracy [4] [9].

In Section 2, this paper first defines key terminology used throughout the paper. Section 3 presents a taxonomy which contextualizes this work in the broader field of adversarial robustness and provides an overview of the main papers this survey covers. Section 4 describes adversarial attacks and defenses as background. Sections 5 and 6 describe methods to approximate lower and upper bounds for RDs respectively, including a discussion of their use in obtaining RDs and their limitations. In particular, the formal verifiers $\alpha, \beta-$CROWN [31] and Branch-and-Bound (BaB) [5], and the adversarial attacks Fast Gradient Sign Method (FGSM) [14] and the more advanced Projected Gradient Descent (PGD) [21] will be discussed. Finally, Section 7 discusses the applications of RDs, and Section 8 highlights current open challenges.

2 Definitions

The most important definitions used in this survey are defined below.

Epsilon ε The perturbation size of an adversarial attack, measured in a defined ℓ-norm. This attack can be either successful or unsuccessful.

Lower and upper bounds Four notions of lower and upper bounds are discussed in Table 1.

At the heart of robustness distributions lies the gap between provable robustness and realized adversarial vulnerability. Complete verification methods provide a lower-bound estimate $\tilde{\varepsilon^*}$ of the critical robustness radius ε^*, whereas either complete verifiers or other methods such as adversarial attacks provide an upper-bound estimate $\tilde{p^*}$ of the minimum adversarial distortion p^*. Consequently, the following inequality holds:

$$\tilde{\varepsilon^*} \leq \varepsilon^* < p^* \leq \tilde{p^*}. \tag{1}$$

Verification gap The difference between the empirical upper bound $\tilde{p}^*$ and the empirical lower bound $\tilde{\varepsilon}^*$. In theory, the true ε^* and p^* only differ from each other by an infinitesimal amount, but in practice search is discretized using a step size, and time-outs and out-of-memory errors can occur by the verifier due to complexity on certain ε leading to verification gaps.

Symbol	Name	Description
ε^*	Critical epsilon	The maximum perturbation ε that *cannot* lead to an adversarial example such that any larger ε can provably lead to one [4].
$\tilde{\varepsilon^*}$	Estimated critical epsilon	An empirical estimation of ε^*.
p^*	Minimum adversarial distortion	The smallest amount of perturbation that is necessary to create an adversarial example that can mislead a network, such that any smaller perturbation could not lead to a misclassification for a given instance [4].
$\tilde{p^*}$	Estimated minimum adversarial distortion	An empirical estimation of p^*.

Table 1: Overview of lower and upper bounds used in formal network robustness verification.

Complete verifier A verifier is complete if an adversarial example is guaranteed to exist at ε when the verifier returns "not verified". Conversely, an incomplete verifier does not guarantee whether an adversarial example exists at ε when it returns "not verified" [18].

Soundness A verifier is sound if every time it returns "verified", it guarantees that no adversarial example exists at ε. Conversely, a verifier is unsound if, when it returns "verified", it does not guarantee whether or not an adversarial examples exists [34].

Robustness in an image classification model is defined as

$$\forall x_0 : ||x - x_0||_p \leq \varepsilon \Rightarrow f(x) = f(x_0), \tag{2}$$

where for any observation x and model f, small perturbations no larger than ε (within a specific ℓ_p-norm as seen in Section 4.2) will not lead to a different classification [12]. If $f(x) \neq f(x_0)$, x is an "adversarial example".

3 Taxonomy

The taxonomy of robustness in NNs is visualized in Figure 1. Robustness of NNs can be viewed through different lenses such as perturbation budget curves [8, 6]. For this survey, RDs are identified as one particular lens—or concrete tool to analyze the robustness of NNs.

RDs show for each input to the NN an estimated lower bound $\tilde{\varepsilon^*}$ and upper bound $\tilde{p^*}$ under some ℓ-norm. While formal complete verifiers such as α, β-CROWN and BaB can approximate both, they are computationally expensive, so relatively cheap adversarial attack-based methods using e.g., FGSM and PGD are often used to estimate upper bounds.

These methods can be used to obtain RDs which in turn can be used to to analyze, for example, the effects of adversarial training and the robustness of different NN architectures. This analysis can provide guidance for improving training procedures and defenses.

The seven papers this survey covers in detail are visualized in the taxonomy. The paper by Bosman et al. [4] zooms in on formal verification methods, using complete verifiers to obtain approximations of ε^* and p^*. Contrarily, Liu et al. [20] and Berger et al. [3] discuss a computationally cheaper way to obtain approximations of p^* using adversarial attacks. We zoom in on the two most popular complete formal verification methods α, β-CROWN [31] and BaB [5] which can approximate ε^* and p^*, and two popular adversarial attacks FGSM [14] and PGD [21] to approximate p^*. Finally, we briefly discuss the applications of RDs.

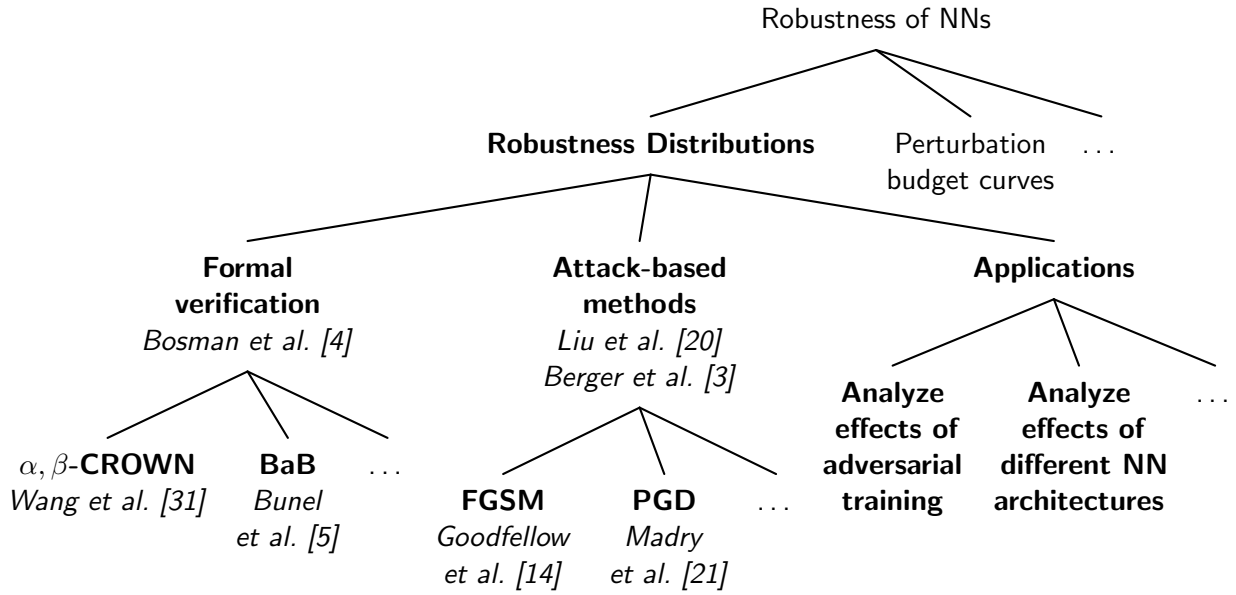

Figure 1: RDs are one lens—or tool—for analyzing the robustness of a NN. This lens is what this survey focuses on.

4 Background

This section discusses adversarial attacks on NNs, their defenses, and the other lenses to view the robustness against these attacks.

4.1 Adversarial attacks and defenses

As NNs are becoming increasingly omnipresent in critical areas such as autonomous driving [22, 26], malware detection [32, 29], and healthcare [33, 19], the interest of malicious actors to interfere in the decision making of these models has risen.

4.1.1 Adversarial attacks

The goal of an adversarial attack on a NN is to alter correct NN predictions into erroneous ones, by manipulating the input data with small imperceptible changes that would not change human prediction. The most widely discussed conceptualization in literature, is one of image classification. In this context, the goal of an adversarial attack would be to find a small imperceptible perturbation in pixel values for an image such that the model misclassifies a previously correctly classified image, while a human's prediction would not change. Such an image is an "adversarial example". This phenomenon can have significant real-world consequences, both when adversaries intentionally introduce noise, and with unintentionally captured noise resulting from imperfect image signals. Real-world implications include: misinterpretation of traffic signs in autonomous vehicles, failed human/object recognition by surveillance and security systems, misclassifications of medical image data leading to erroneous medical diagnoses, and fraud in identity verification for device unlocks and payments.

4.1.2 Adversarial defenses

To protect NNs from these adversarial attacks, various defenses have been proposed. An adversarial defense aims to improve the robustness of the model by ensuring a correct prediction remains, even if the input is slightly perturbed. Note that the performance of initially incorrect predictions is therefore not (necessarily) improved.

4.2 Perturbation metrics

To measure the magnitude of perturbations, ℓ-norms are used [7]. In literature, four different types are distinguished, summarized in Table 2.

	Description	Given original input x_0 and perturbed input x
ℓ_0**-norm** *i.e. Sparsity norm*	The number of individual features modified, regardless of the magnitude of the change. In an image this would be the number of pixels that were modified.	$\|\|x_0 - x\|\|_0 = \sum_i \mathbb{I}[x_{i,0} \neq x_i]$
ℓ_1**-norm** *i.e. Manhattan norm*	The sum of the absolute changes across all features.	$\|\|x_0 - x\|\|_1 = \sum_i \|x_{i,0} - x_i\|$
ℓ_2**-norm** *i.e. Euclidean norm*	The Euclidean distance between the original and perturbed input.	$\|\|x_0 - x\|\|_2 = \sqrt{\sum_i (x_{i,0} - x_i)^2}$
ℓ_∞**-norm** *i.e. Maximum norm*	The difference of the single largest changed feature.	$\|\|x_0 - x\|\|_\infty = \max_i \|x_{i,0} - x_i\|$

Table 2: Overview of the four different ℓ-norms used in NN robustness analysis.

4.3 Other lenses of robustness

This section briefly touches upon other lenses through which robustness of NNs can be viewed.

4.3.1 Metrics

Over the years, many metrics have been proposed to capture the robustness of a NN for measurement and comparison, resulting in a scattered landscape of measures and metrics [9, 30, 24, 23, 28]. The commonality between all popular metrics is their global scoring: given a dataset, a single global robustness value for the network is given, potentially masking local vulnerabilities for specific inputs.

The most common robustness metric in literature is *Robust accuracy*, defined as the ratio of inputs from a test set that are classified correctly given the perturbation ε [4] [9] . The metric is computed by generating adversarial examples via heuristics (like FGSM or PGD, refer to Section 6.1 and 6.2 respectively), then evaluating classification accuracy on those examples. This results in a single robustness value for a given test set and ε combination. Note that inputs that would result in a misclassification for a smaller perturbation are not distinguished from inputs that would result in a misclassification for a larger perturbation, considering both perturbations are smaller than ε. When no perturbations are applied to the input images (i.e. $\varepsilon = 0$), the metric conveys the traditional notion of model accuracy and is referred to as *clean accuracy* instead.

4.3.2 Perturbation budget curves

The concept of perturbation budget refers to the maximum input perturbation that may be added to the input under a specific ℓ-norm [8, 6]. Perturbation budget curves plot a model's performance (usually robust accuracy) as a function of perturbation budget. This lens addresses the issue that a single robust accuracy value can be misleading, as the shape of the curve reveals whether a method offers narrow or broad protection across perturbation scales.

5 Formal verification

Complete verification provides both provable approximations of *lower bounds* and *upper bounds*; $\tilde{\varepsilon^*}$ and $\tilde{p^*}$ respectively. In the context of classification, the objective of complete verification is to prove the following:

$$\forall x \in \mathcal{S},\ \ \mathcal{M}(x) = y_0(x) - y(x) > 0, \tag{3}$$

where y_0 denotes the correct ("source") class, y' is any other class, and $\mathcal{S}$ is the input domain. Note that proving this is equivalent to proving that $\forall x \in \mathcal{S}$, $y_0(x) > y'(x)$ (i.e., the source class has the greatest logit). So, the objective function is $\mathcal{M}(x)$ and an adversarial example is an input x such that $\mathcal{M}(x) < 0$. Finding the minimum of $\mathcal{M}(x)$ is challenging because $\mathcal{M}(x)$ is a function of $y(x)$ (the output of the NN) and NNs are generally non-convex.

5.1 Branch and Bound

Branch and Bound (BaB) is an algorithm that minimizes an objective function by pruning a search space. In the context of verification the objective function is $\mathcal{M}(x)$, as shown in Equation 5 [5]. To minimize $\mathcal{M}(x)$, BaB first uses an incomplete verifier, for example a linear programming (LP) solver, to compute an upper and lower bound on the network [18]. If the upper bound is negative, the algorithm terminates as "not verified" (this provides a lower bound for ε^*); if the lower bound is positive the algorithm terminates as "verified" (this provides an upper bound for ε^*) [18]. If neither condition is true, BaB splits the network at an activation function ("branching") into two copies of the network, one in which the ReLU is active and one in which it is not. (Here, BaB relies on the piecewise linearity of ReLU-only NNs [5].) It uses incomplete linear solvers to solve the resulting linear networks [5]. The algorithm terminates if, for both copies, $\mathcal{M}(x)$ is always positive/negative. Otherwise, branching is performed again and the process repeats [18]. BaB is a complete verification method [18].

5.2 α, β-CROWN

CROWN is an incomplete formal verification method in which a linear lower bound is created for $\mathcal{M}(x)$ [31]. Linear inequalities are propagated through the network (similar to how gradients would be in gradient descent). If the linear bound is greater than 0 for all x in the domain, then it is verified that $\mathcal{M}(x)$ is greater than 0 for all x in the domain. Non-linear (e.g., ReLU) behavior is dealt with by using linear bounds over a restricted domain to approximate the linear function (figure 2). Either the upper or lower bound on this region is propagated, depending on whether the coefficient of the ReLU is negative or positive respectively [31]. This is repeated until the input layer is reached and this yields a linear bound for the function $\mathcal{M}(x)$ on the input domain. The propagation of inequalities

is fast and can be accelerated on GPUs [31]. Variations have been built on CROWN to make the bounds as tight as possible. One such variation is β-CROWN, in which CROWN is combined with BaB and parameters $\boldsymbol{\beta}$ are optimized with gradient ascent to tighten the lower bound on $\mathcal{M}(x)$ [31]. β-CROWN is a complete verifier with three order of magnitude speedup compared to BaB with LP verifiers [31]. Another variation, α,β-CROWN (also complete), builds on β-CROWN by using free parameters $\boldsymbol{\alpha}$ to optimize the slope of the linear bounds (figure 2) to further tighten bounds [31].

5.3 Other verifiers

Matthias et al. [17] distinguish between CPU- and GPU-based complete verification methods. Examples of CPU-based methods include BaB, Marabou, Neurify, nnenum, and VeriNet. Marabou is a tool based on SMT (satisfiability modulo theories); it answers queries about the network by solving constraint satisfaction problems and it accomodates ReLU and Sigmoid activation functions [16]. The nnenum method [2] uses zonotopes and star sets, two representations of sets of states to quickly and accurately verify ReLU networks.

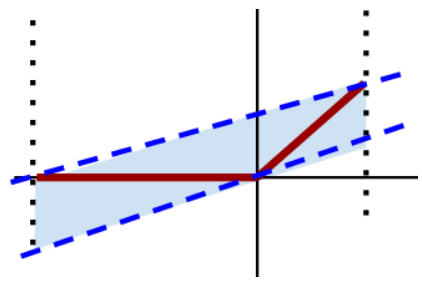

Figure 2: Local approximation of a function in a non-linear region using linear bounds [31].

GPU-based methods include BaDNB, β-CROWN, and MN-BaB. BaDNB (Branch and Dual Network Bound) is a framework based on novel bounding algorithms and a novel activation-based branching strategy presented by [27]. MN-BaB (Multi-Neuron Constraint Guided BaB) combines the BaB framework with multi-neuron constraints from [25] to achieve state-of-the-art performance [13].

5.4 Limitations

Complete verifiers are computationally expensive in time and memory resources. Often complete verifiers result in time-out or out-of-memory errors. This results in RDs with large verification gaps from which it is difficult to draw conclusions [4] .

Complete verifiers are also limited in the architectures for which they can provide verifications. This is because a NN must be formulated as a mixed-integer programming (MIP) problem to be solved using a complete verifier. This is not a theoretical constraint, but a practical one. Many operations in NNs are missing "translations" to the MIP context. As the field grows, it will be possible to apply complete verifiers to more NNs than currently.

5.5 Link to RDs

Complete verifiers can be used to obtain RDs by searching for counterexamples to $\mathcal{M}(x) < 0$ for a given value of epsilon [4]. The upper bounds for p^* are correct because of the definition of completeness (see section 2). However, it has been shown that complete verifiers are not always sound, so lower bounds are not guaranteed to be correct [35, 15]. Formal methods that estimate both upper and lower bounds on ε^* may be used to obtain verification gaps which show a range within which ε^* exists for a network-instance pair [4, 3].

6 Attack-based approximation of p^*

The aforementioned methods provide *lower bounds* and *upper bounds*; $\tilde{\varepsilon^*}$ and $\tilde{p^*}$ respectively. These methods, however, are computationally expensive, scaling poorly with network and dataset size [4]. For more efficient robustness approximations of p^*, attack-based methods are considered to compute empirical *upper bounds*, which are computationally much cheaper. However, they do not provide any guarantees of finding adversarial examples in certain radii even if they do exist.

6.1 Fast Gradient Sign Method

Some techniques rely on first-order information of the loss $L(\theta, x, y)$: Fast Gradient Sign Method (FGSM) [14] computes a one-step perturbation in the direction of the steepest increase of the loss:

$$x = x_0 + \varepsilon \,\mathrm{sign}(\nabla_x L(\theta, x_0, \lambda(x_0))). \tag{4}$$

x would be an adversarial example if it would cause the model to misclassify, breaking the *robustness* definition seen in eq. 2. The gradient $\nabla_x L(\theta, x_0, \lambda(x_0))$ indicates the direction in input space that locally increases the loss most rapidly. By taking the element-wise sign of this gradient and scaling it by ε, FGSM computes a perturbation that maximizes the first-order increase of the loss under an ℓ_∞-norm constraint, yielding a one-step approximation of a worst-case adversarial perturbation.

The parameter ε controls the magnitude of the perturbation applied to the input by bounding the maximum change of each pixel value. Under the ℓ_∞-norm, each pixel is independently perturbed by ε in the direction that increases the loss. Larger values of ε therefore result in stronger and potentially visible perturbations, whereas smaller values may be insufficient to cause misclassification.

6.2 Projected Gradient Descent

FGSM is limited due to it's single-step maximization, and therefore often fails to find minimally adversarial perturbations p^*, resulting in conservative upper-bound approximations. To address this limitation, Projected Gradient Descent (PGD) [21] is considered which performs iterative loss maximization.

Expanding on FGSM, starting from an initial perturbed input x_0, PGD repeatedly applies small gradient-ascent steps to increase the loss, followed by a projection that enforces an ℓ_∞-norm constraint:

$$x_{t+1} = \Pi_{\mathcal{B}_\infty(x_0,\varepsilon)}(x_t + \alpha \,\mathrm{sign}(\nabla_x L(\theta, x_t, \lambda(x_0)))) \,, \;\; \mathcal{B}_\infty(x_0, \varepsilon) = \{x | x - x_0 < \varepsilon\} \tag{5}$$

where α denotes the step size and $\Pi_{\mathcal{B}_\infty(x_0,\varepsilon)}$ projects the updated input back onto the ℓ_∞-ball of radius ε if the step falls outside of this ball, centered from the original input x_0. The ℓ_∞-norm constraint ensures that each pixel value is perturbed by at most ε. Adversarial attacks can be embedded within a binary search over perturbation magnitudes ε. In practice, ε is often selected from a discretized search space, for example: $\mathcal{S} := \left\{ \frac{i}{255} \;\middle|\; i \in \mathbb{N} \right\} \cap [0, 0.4]$, although other discretizations or continuous search strategies may also be employed depending on the dataset and perturbation model.

PGD is particularly effective because it approximates the inner maximization problem of robust optimization iteratively to find global optima. Empirically, this gives tight upper-bound approximations of the minimal adversarial distortion p^* [3].

As a result, PGD is widely regarded as the state-of-the-art attack-based method for approximating upper bounds on the critical robustness radius.

6.3 Other attack-based methods

Beyond FGSM and PGD, several other attacks are frequently used to approximate the smallest perturbation that can cause misclassification.

One family of techniques attempts to move directly toward the decision boundary. The FAB attack [10] is a prominent example: it iteratively adjusts candidate perturbations with the goal of reducing their size. Other approaches, such as AutoAttack [11], combine multiple strong attacks into an ensemble designed to increase reliability and reduce dependency on hyperparameter tuning. Even black-box strategies have been attempted: the Square Attack [1] rely on structured random search to discover weaknesses that gradient-based methods may overlook, making them complementary even when they lack the precision of boundary-focused attacks.

6.4 Limitations

Despite their diversity, all of these approaches remain empirical attacks: they cannot guarantee finding an adversarial example when one exists nor can they provide provable tightest upper-bounds. They provide no certificates of robustness. Consequently, the perturbation levels they discover should be interpreted solely as empirical upper bounds on the true critical value p^*, rather than definitive estimates. Failure to find an adversarial example cannot be taken as evidence of robustness, limiting the suitability of these methods for formal verification tasks even when they offer substantial practical value for robustness evaluation.

6.5 Link to RDs

Adversarial attacks play a central role in RDs. Their main contribution lies in enabling the computationally inexpensive approximation of *upper bounds* on the minimum adversarial distortion p^* which can be paired with the critical value ε^* explained in the inequality 2. As demonstrated by Berger et al. [3], strong adversarial attacks can give considerably tight upper-bound approximations of p^*, allowing approximate RDs to be constructed even for larger models where exact verification using complete verifiers is not feasible.

7 The applications of RDs

RDs were introduced to address fundamental limitations of traditional robustness metrics. Robust accuracy reports only a single value at a fixed perturbation size, not revealing how robustness varies across individual samples in the dataset, while instance-level metrics fail to showcase the model's robustness. Instead, considering the ε^* over many inputs, RDs such as box plots and empirical cumulative distribution functions (ECDFs) can be generated.

With complete verification RDs can be generated. They illustrate nuances when evaluating architectures, adversarial training, and datasets. Bosman et al. [4] show that adversarial training does more than increase robustness: it reshapes the entire RD, shifting and narrowing it in ways that are visible in ECDFs and box plots but invisible to aforementioned metrics. They also observe that certain inputs have low critical values across many networks, suggesting difficulty in datasets rather than verifiers. Additionally, they find that the distributions for train and test sets are often similar, indicating that robustness behaves like a property of the data rather than a side effect of training.

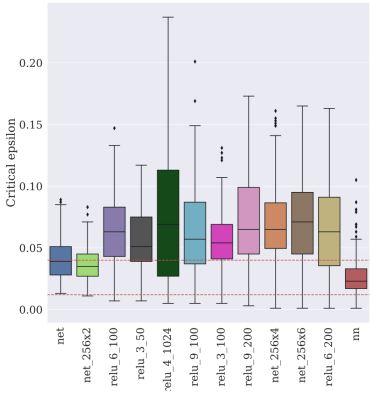

Figure 3: RD with a box plot [4].

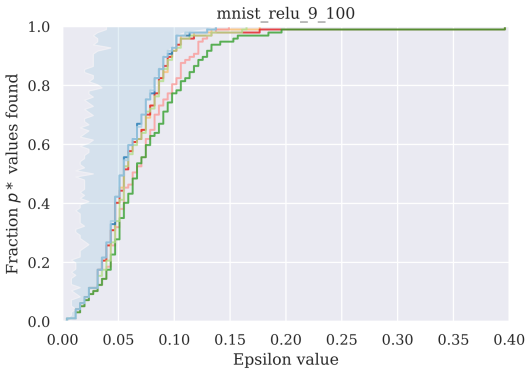

Figure 4: RD with an ECDF [3].

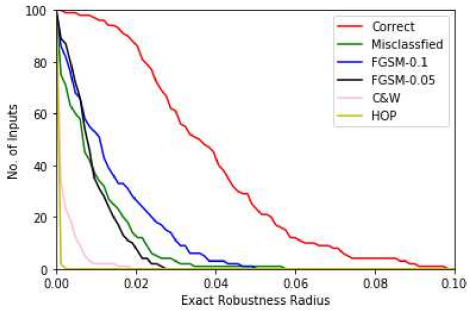

Figure 5: Liu et al. [20] Robustness Radius illustration.

Adversarial attacks enable RD analysis with models that complete verification cannot handle. Adversarial attacks provide empirical *upper bounds* on ε^*, making it possible to estimate approximate RDs when exact values are infeasible. Berger et al. [3] highlights that RDs have *verification gaps*, defined as the difference between the empirical upper bound $\tilde{\varepsilon}^*$ obtained from complete verifiers and the empirical lower bound $\tilde{p}^*$ (obtained through various methods like (in)complete verifiers or adversarial attacks). The verification gap defined as $\tilde{p}^* - \tilde{\varepsilon}^*$, as described in Section 4, indicates how much of the robustness landscape remains unresolved. Visualizing these gaps across the distribution helps identify precisely which regions of the data or which architectures impose the greatest challenge on NN verification.

Visual examples for a box plot and ECDF can be seen in Figures 3 and 4 respectively. The blue area in the latter describes the *verification gap*. These visual tools reveal nuances that other metrics (such as robust accuracy) cannot capture. When deciding which NN to use for an application, one could choose a model with a higher median ε^*, but larger variance (i.e. large fluctuations between the robustness of different inputs). Contrarily, for certain applications one might prefer a model with a lower median ε^* with smaller variance. Surprisingly, these distributions hold a log-normal distribution [4].

Finally, approximate robustness measures, as seen in Liu et al. [20], also utilizes adversarial attacks to estimate robustness radii to distinguish clean from adversarial inputs on small networks, showcasing that these radii cluster differently across input types, with similar Robustness Distribution charts as seen in Figure 5. Although such approaches do not generalize reliably to larger models, they illustrate how RDs can contribute when certified verification is infeasible. Altogether, RDs provide a flexible framework for interpreting robustness information, enabling both scientific analysis and practical applications beyond those offered by traditional robustness metrics.

8 Open challenges

Robustness Distributions (RDs) provide a visual representation of neural network (NN) robustness, yet several aspects remain challenges.

Reducing Verification Gaps for Practical Deployment A primary challenge is the significant computational cost of complete verification, which often leads to a verification gap. On complex datasets like CIFAR-10, this gap can be large [3, 20]. Future work could focus on narrowing this gap to ensure RDs provide a reliable basis for safety-critical deployment.

Ranking Networks While [4] use the mean of RDs to rank networks, a clear framework for using RDs as a multi-objective decision tool is missing. Other statistics, such as the minimum or the variance, are equally relevant. A significant open question is how to balance the trade-off between a network with a high mean RD but a large verification gap versus a network with a lower mean but a tighter, more certain gap.

RDs Beyond Image Classification Most current RD research is limited to image classification (e.g., MNIST, CIFAR-10)[3, 20, 4]. Extending RDs to other domains remains largely unexplored. Each domain presents unique challenges for defining the "critical epsilon" values and finding suitable robustness properties to characterize the distribution effectively.

9 Conclusions

This survey reviews NN robustness verification through the lens of RDs, a niche method for evaluating adversarial robustness in NNs. RDs show distributions of perturbations across individual inputs, with a turning point known as the critical epsilon ε^*. Experiments revealed that critical epsilons follow a log normal distribution; a behavior that is hidden by traditional metrics such as robust accuracy.

Both complete verifiers and attack-based methods are reviewed in this survey as methods to obtain the RDs. Complete verifiers provide certified lower bounds and upper bounds, but suffer from scalability. To compensate, adversarial attacks are introduced to more efficiently approximate upper bounds, although they do not guarantee finding adversarial examples in certain radii even if they do exist. Using both the lower bound and upper bound for each input, RDs are visualized in various ways including as box plots and ECDFs.

RDs are applicable as they offer visual insights into the adversarial robustness of NNs for each input of a dataset which are not apparent using other lenses or metrics. For example, they showcase verification gaps that highlights the limitations of the respective datasets in combination with the utilized (in)complete verifiers and attacks.

Areas of future work could include expanding complete verification methods to support more NN architectures, developing frameworks for ranking NNs using RDs, and RDs beyond image classification.

References

[1] Maksym Andriushchenko, Francesco Croce, Nicolas Flammarion, and Matthias Hein. Square attack: a query-efficient black-box adversarial attack via random search. *CoRR*, abs/1912.00049, 2019.

[2] Stanley Bak. nnenum: Verification of relu neural networks with optimized abstraction refinement. In Aaron Dutle, Mariano M. Moscato, Laura Titolo, César A. Muñoz, and Ivan Perez, editors, *NASA Formal Methods - 13th International Symposium, NFM 2021, Virtual Event, May 24-28, 2021, Proceedings*, volume 12673 of *Lecture Notes in Computer Science*, pages 19–36. Springer, 2021.

[3] Aaron Berger, Nils Eberhardt, Annelot Willemijn Bosman, Henning Duwe, Jan N van Rijn, and Holger Hoos. Empirical analysis of upper bounds of robustness distributions using adversarial attacks. In *The 19th learning and intelligent optimization conference*, 2025.

[4] Annelot W. Bosman, Aaron Berger, Holger H. Hoos, and Jan N. van Rijn. Robustness distributions in neural network verification. *Journal of Artificial Intelligence Research*, 83, 2025.

[5] Rudy Bunel, Ilker Turkaslan, Philip H. S. Torr, Pushmeet Kohli, and Pawan Kumar Mudigonda. A unified view of piecewise linear neural network verification. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 4795–4804, 2018.

[6] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian J. Goodfellow, Aleksander Madry, and Alexey Kurakin. On evaluating adversarial robustness. *CoRR*, abs/1902.06705, 2019.

[7] Nicholas Carlini and David A. Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 39–57. IEEE Computer Society, 2017.

[8] Antonio Emanuele Cinà, Maura Pintor, Luca Demetrio, Ambra Demontis, Battista Biggio, and Fabio Roli. Evaluating the evaluators: Trust in adversarial robustness tests. *CoRR*, abs/2507.03450, 2025.

[9] Joana C. Costa, Tiago Roxo, Hugo Proença, and Pedro Ricardo Morais Inácio. How deep learning sees the world: A survey on adversarial attacks defenses. *IEEE Access*, 12:61113–61136, 2024.

[10] Francesco Croce and Matthias Hein. Minimally distorted adversarial examples with a fast adaptive boundary attack. *CoRR*, abs/1907.02044, 2019.

[11] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. *CoRR*, abs/2003.01690, 2020.

[12] Yinpeng Dong, Qi-An Fu, Xiao Yang, Tianyu Pang, Hang Su, Zihao Xiao, and Jun Zhu. Benchmarking adversarial robustness on image classification. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pages 318–328. Computer Vision Foundation / IEEE, 2020.

[13] Claudio Ferrari, Mark Niklas Müller, Nikola Jovanovic, and Martin T. Vechev. Complete verification via multi-neuron relaxation guided branch-and-bound. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.

[14] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[15] Kai Jia and Martin C. Rinard. Exploiting verified neural networks via floating point numerical error. In Cezara Dragoi, Suvam Mukherjee, and Kedar S. Namjoshi, editors, *Static Analysis - 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17-19, 2021, Proceedings*, volume 12913 of *Lecture Notes in Computer Science*, pages 191–205. Springer, 2021.

[16] Guy Katz, Derek A. Huang, Duligur Ibeling, Kyle Julian, Christopher Lazarus, Rachel Lim, Parth Shah, Shantanu Thakoor, Haoze Wu, Aleksandar Zeljic, David L. Dill, Mykel J. Kochenderfer, and Clark W. Barrett. The marabou framework for verification and analysis of deep neural networks. In Isil Dillig and Serdar Tasiran, editors, *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part I*, volume 11561 of *Lecture Notes in Computer Science*, pages 443–452. Springer, 2019.

[17] Matthias König, Annelot W. Bosman, Holger H. Hoos, and Jan N. van Rijn. Critically assessing the state of the art in neural network verification. *J. Mach. Learn. Res.*, 25:12:1–12:53, 2024.

[18] Linyi Li, Tao Xie, and Bo Li. Sok: Certified robustness for deep neural networks. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1289–1310. IEEE, 2023.

[19] Znaonui Liang, Gang Zhang, Jimmy Xiangji Huang, and Qmming Vivian Hu. Deep learning for healthcare decision making with emrs. In *2014 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 556–559. IEEE, 2014.

[20] Jiangchao Liu, Liqian Chen, Antoine Miné, and Ji Wang. Input validation for neural networks via runtime local robustness verification. *CoRR*, abs/2002.03339, 2020.

[21] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *CoRR*, abs/1706.06083, 2017.

[22] Ana I. Maqueda, Antonio Loquercio, Guillermo Gallego, Narciso Garcia, and Davide Scaramuzza. Event-based vision meets deep learning on steering prediction for self-driving cars. In *CVF Conference on Computer Vision and Pattern Recognition*, page 5419–5427. IEEE, June 2018.

[23] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Deepfool: A simple and accurate method to fool deep neural networks. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 2574–2582. IEEE Computer Society, 2016.

[24] Konda Reddy Mopuri, Vaisakh Shaj, and R. Venkatesh Babu. Adversarial fooling beyond "flipping the label". *CoRR*, abs/2004.12771, 2020.

[25] Mark Niklas Müller, Gleb Makarchuk, Gagandeep Singh, Markus Püschel, and Martin T. Vechev. PRIMA: general and precise neural network certification via scalable convex hull approximations. *Proc. ACM Program. Lang.*, 6(POPL):1–33, 2022.

[26] Anselme Ndikumana, {Nguyen H.} Tran, {Do Hyeon} Kim, {Ki Tae} Kim, and {Choong Seon} Hong. Deep learning based caching for self-driving cars in multi-access edge computing. *IEEE Transactions on Intelligent Transportation Systems*, 22(5):2862–2877, May 2021.

[27] Alessandro De Palma, Rudy Bunel, Alban Desmaison, Krishnamurthy Dvijotham, Pushmeet Kohli, Philip H. S. Torr, and M. Pawan Kumar. Improved branch and bound for neural network verification via lagrangian decomposition. *CoRR*, abs/2104.06718, 2021.

[28] Wenjie Ruan, Min Wu, Youcheng Sun, Xiaowei Huang, Daniel Kroening, and Marta Kwiatkowska. Global robustness evaluation of deep neural networks with provable guarantees for the hamming distance. In Sarit Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 5944–5952. ijcai.org, 2019.

[29] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, and Sitalakshmi Venkatraman. Robust intelligent malware detection using deep learning. *IEEE Access*, 7:46717–46738, 2019.

[30] Jie Wang, Jun Ai, Minyan Lu, Haoran Su, Dan Yu, Yutao Zhang, Junda Zhu, and Jingyu Liu. A survey of neural network robustness assessment in image recognition. *CoRR*, abs/2404.08285, 2024.

[31] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 29909–29921, 2021.

[32] Zhenlong Yuan, Yongqiang Lu, Zhaoguo Wang, and Yibo Xue. Droid-sec: deep learning in android malware detection. *SIGCOMM Comput. Commun. Rev.*, 44(4):371–372, August 2014.

[33] Xiaokang Zhou, Wei Liang, Kevin I-Kai Wang, Hao Wang, Laurence T. Yang, and Qun Jin. Deep-learning-enhanced human activity recognition for internet of healthcare things. *IEEE Internet of Things Journal*, 7(7):6429–6438, 2020.

[34] Xingjian Zhou, Keyi Shen, Andy Xu, Hongji Xu, Cho-Jui Hsieh, Huan Zhang, and Zhouxing Shi. Soundnessbench: A soundness benchmark for neural network verifiers. *Transactions on Machine Learning Research*, December 2025.

[35] Dániel Zombori, Balázs Bánhelyi, Tibor Csendes, István Megyeri, and Márk Jelasity. Fooling a complete neural network verifier. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.